T3.7 Multi Script Router: Two-Stage CNN for Multi-Script Indian Handwritten Character Recognition

Shivam Patel
shivam.p@students.iiit.ac.in
Roll No: 2025202030

Kavan Gandhi
kavan.gandhi@students.iiit.ac.in
Roll No: 2025201078

Suyash Pande
suyash.pande@students.iiit.ac.in
Roll No: 2025201063

Dhruv Rajput
dhruv.rajput@students.iiit.ac.in
Roll No: 2025201094

Divyanshu Jain
divyanshu.jain@students.iiit.ac.in
Roll No: 2025201059

Aayush Pandey
aayush.pandey@students.iiit.ac.in
Roll No: 2025201058

Nikhil Patidar
nikhil.patidar@students.iiit.ac.in
Roll No: 2025201081

Abstract

We present a two-stage pipeline for recognising handwritten characters across four major Indian scripts: Devanagari, Tamil, Bengali (BanglaLekha), and Kannada. The first stage is a lightweight Router-CNN that predicts the input script from a 32×32 grayscale image; the second stage routes the image to the corresponding script-specific classifier trained on dedicated benchmark datasets (UCI Devanagari, uTHCD Tamil, BanglaLekha-Isolated, and Kannada-MNIST).

All models are compact CNNs trained from scratch, collectively covering 296 character classes. The full system is deployed as an interactive Streamlit web application hosted on Hugging Face Hub, supporting both free-draw and image-upload inference, as well as a practice mode for learners.

Our router achieves **99.03%** test accuracy on the four-class script-identification task, while script-specific classifiers attain top-1 accuracies of: **99.36%** (Devanagari), **97.7%** (Tamil), **93.4%** (Bengali), and **87.99%** (Kannada).

Keywords

handwritten character recognition, multi-script, convolutional neural network, Indian scripts, script identification

1 Introduction

Handwritten character recognition (HCR) for Indian scripts is a long-standing challenge in document analysis due to the linguistic diversity of the subcontinent: India recognises 22 official languages written in at least 12 distinct scripts, with individual scripts often containing hundreds of base characters and compound ligatures. Existing systems typically address a single script in isolation, which limits practical utility in multilingual environments such as postal sorting, archival digitisation, and educational technology.

This work targets four high-resource scripts simultaneously: **Devanagari** (Hindi, Marathi; 46 classes including digits), **Tamil** (156 classes covering vowels, consonants, and compounds), **Bengali** (84 classes from BanglaLekha-Isolated), and **Kannada** (10 digit classes from Kannada-MNIST). Together these scripts cover approximately 600 million native speakers.

We make three main contributions:

(1) A *two-stage inference pipeline* in which a shared router identifies the script before delegating to the specialised classifier, cleanly separating script-level and character-level uncertainty.
(2) Four compact CNN classifiers trained from scratch on publicly available benchmarks, collectively covering 296 character classes with no pre-training or external features.
(3) An end-to-end *deployed interactive web application* (Streamlit + Hugging Face Hub) supporting draw, upload, and gamified practice modalities.

2 Related Work

Single-script recognition. Acharya *et al.* [1] introduced the UCI Devanagari benchmark and demonstrated CNNs superior to SVM baselines. The uTHCD Tamil database [2] remains the largest publicly available unconstrained Tamil handwriting corpus at ∼91k images across 156 classes. BanglaLekha-Isolated [3] provides 166k images of Bengali characters collected under varying writing conditions. Prabhu [4] released Kannada-MNIST as a drop-in alternative to MNIST with strong CNN baselines exceeding 98%.

Multi-script recognition. Multi-script document analysis has been explored using handcrafted features (Gabor filters, gradient features) [5] and, more recently, via convolutional feature pooling and attention mechanisms [6]. However, most prior work tackles script *identification at the document or word level* rather than at the *isolated character level*, the latter being harder because individual strokes share visual motifs across scripts.

Deployed systems. While many papers report benchmark numbers, publicly accessible interactive demos remain scarce. Our work follows the spirit of [7] in bridging research and deployment by serving live models via a web interface.

3 Data

Table 1 summarises the four benchmark datasets used. All images are resized to 32×32 grayscale for a unified input representation across the pipeline.

Table 1: Dataset statistics. "Classes" counts distinct character categories; "Images" is the total before train/val/test split.

Dataset	Script	Classes	Images	Split
UCI Devanagari [1]	Devanagari	46	92,000	80/20 train/test
uTHCD [2]	Tamil	156	91,000	90/10 (split d)
BanglaLekha-Iso. [3]	Bengali	84	166,105	75/15/10
Kannada-MNIST [4]	Kannada	10	60,000	official
Router (balanced subsample)	All four	4	296,000	70/15/15

Router dataset construction. A balanced router training set is assembled by sampling up to 500 images per character class from each script's training split, then assigning a script-level label $y \in \{0, 1, 2, 3\}$ for Devanagari, Tamil, Bengali, and Kannada respectively. This yields ≈17k training images (post-split) with near-equal class balance (§6).

Pre-processing. Images are converted to single-channel grayscale, resized using bilinear interpolation to 32×32, and normalised to $[-1, 1]$ via $\mu = 0.5$, $\sigma = 0.5$. During training, augmentations include random rotation (±10°), random affine translation (±10%), and optional random perspective distortion ($p = 0.3$, distortion scale 0.2), applied only to training splits.

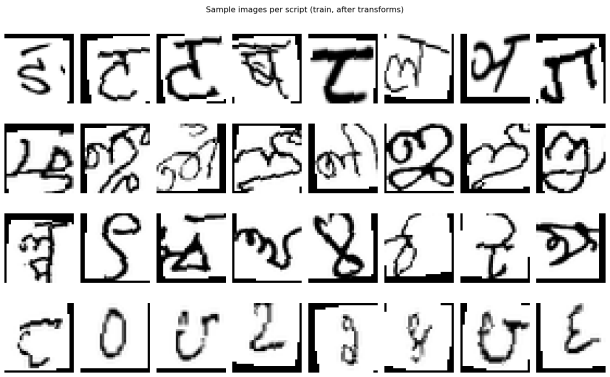


Figure 1: Representative samples from the four benchmark datasets. All images are resized to 32×32 grayscale before training.

4 Method

4.1 Two-Stage Pipeline Overview

Given an input image $\mathbf{x} \in \mathbb{R}^{1\times32\times32}$:

(1) **Stage 1 — Script Router:** $\hat{s} = \arg\max_s f_{\theta_R}(\mathbf{x})$, where f_{θ_R} is the RouterCNN (4-class softmax).
(2) **Stage 2 — Character Classifier:** $\hat{c} = \arg\max_c g_{\theta_{\hat{s}}}(\mathbf{x})$, where $g_{\theta_{\hat{s}}}$ is the script-specific CNN for predicted script $\hat{s}$.
(3) **Label mapping:** $\hat{c}$ is looked up in a Unicode dictionary to produce the rendered character for display.

4.2 RouterCNN

The router is a 3-block convolutional network (Table 2). Each block applies two 3×3 convolutions with Batch Normalisation and ReLU activations, followed by 2×2 max-pooling and spatial dropout. The classifier head is a two-layer MLP with 256 hidden units and 40% dropout. The network outputs a 4-way softmax.

Table 2: RouterCNN architecture. Convolution layers use 3×3 kernels with padding 1. *Drop.* denotes spatial (2D) dropout.

Block	Layer	Channels	Output size
Block 1	Conv-BN-ReLU	32	32×32
	Conv-BN-ReLU	32	32×32
	MaxPool 2×2	32	16×16
	Drop2d (0.10)	32	16×16
Block 2	Conv-BN-ReLU	64	16×16
	Conv-BN-ReLU	64	16×16
	MaxPool 2×2	64	8×8
	Drop2d (0.20)	64	8×8
Block 3	Conv-BN-ReLU	128	8×8
	MaxPool 2×2	128	4×4
Head	Flatten	2048	Vector
	Linear-ReLU	256	Vector
	Dropout (0.40) + Linear	4	Vector

Training details. The router is trained for 20 epochs with AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.999$), an initial learning rate of 3×10^{-3}, weight decay 10^{-4}, and a OneCycleLR scheduler (warm-up 30% of total steps, cosine annealing thereafter). Cross-entropy with label smoothing $\varepsilon = 0.1$ is used as the loss. Batch size is 128; inputs are drawn from the balanced router dataset (§3).

4.3 Script-Specific Classifiers

Devanagari (ModelA, 46 classes). A 3-block CNN with increasing filter widths $64 \to 128 \to 256$, 5×5 then 3×3 kernels, Batch Normalisation, and two fully connected layers (512 units, 50%/30% dropout). Trained with Adam + CosineAnnealingLR for 15 epochs; augmentation: rotation ±10, random affine, random perspective.

Tamil (TamilCNN, 156 classes). A 4-block CNN with widths $32 \to 64 \to 128 \to 256$. Batch Normalisation and 2×2 pooling after each block; the classifier head uses adaptive average pooling followed by two linear layers. Label smoothing $\varepsilon = 0.1$. Trained for 15 epochs on uTHCD split (d) (90/10); validation carved from the 90% train portion at 10%.

Bengali (BengaliCNN, 84 classes). A 3-block CNN with widths $32 \to 64 \to 128$, two 3×3 convolutions per block, followed by global average pooling and a 512-unit classifier head. Trained with OneCycleLR for 20 epochs; dropout 0.4.

Kannada (KannadaCNN, 10 classes). A 3-block CNN with widths $32 \to 64 \to 128$, 5×5 then 3×3 kernels, Batch Normalisation, and a two-layer head (256 units, dropout 0.4). Trained with ReduceLROnPlateau on the Dig-MNIST holdout; augmentation as above.

4.4 Deployment

All five model weights are hosted on Hugging Face Hub and loaded at startup via `hf_hub_download`. The Streamlit application exposes two modes: (i) *Draw & Predict* — a 280×280 freehand canvas with automatic tight-crop and Gaussian blur ($r = 2$) to simulate scanned-image texture; and (ii) *Practice Mode* — a gamified interface that presents a target Unicode character and gives real-time feedback on the user's drawing.

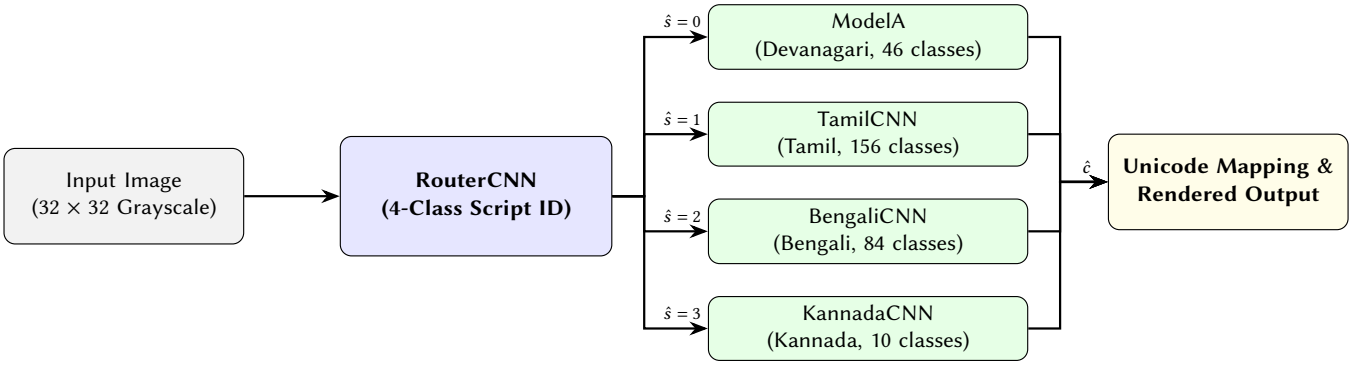


Figure 2: High-level architecture of the two-stage inference pipeline. The input image is first processed by the RouterCNN to identify the script, which dynamically routes the image to the corresponding script-specific CNN. The predicted class $\hat{c}$ is then mapped to its Unicode representation.

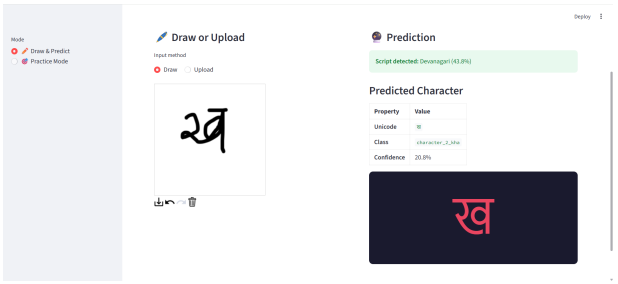


Figure 3: Deployed application. *Left:* Draw & Predict mode; a user's freehand stroke is cropped, blurred, and passed through the two-stage pipeline. *Right:* Practice Mode; the app renders a target Unicode character and grades the user's drawing.

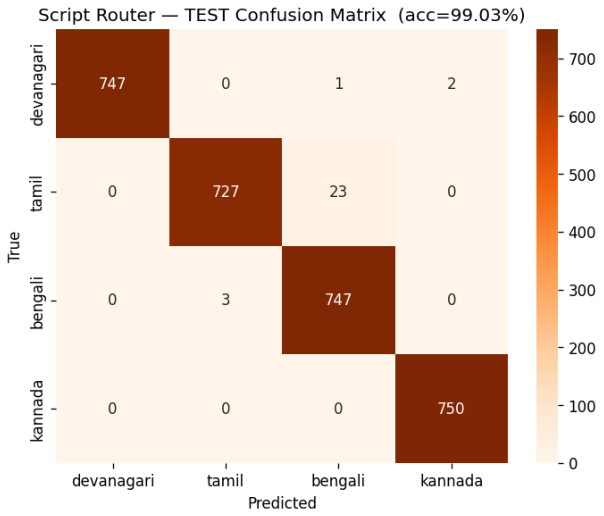


Figure 4: Normalised confusion matrix for the RouterCNN (test set, row-normalised).

5 Results

5.1 Script Router

Table 3 reports per-class and macro-averaged metrics on the held-out router test set (15% of the router dataset, ≈22k images).

Table 3: RouterCNN test-set performance (4-class script identification).

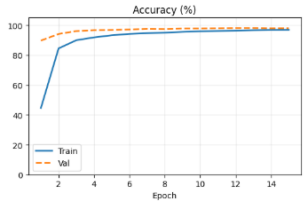
Script	Precision	Recall	F1	Support
Devanagari	1.00	1.00	1.00	750
Tamil	1.00	0.97	0.98	750
Bengali	0.97	1.00	0.98	750
Kannada	1.00	1.00	1.00	750
Macro avg	**0.99**	**0.99**	**0.99**	3000

5.2 Script-Specific Classifiers

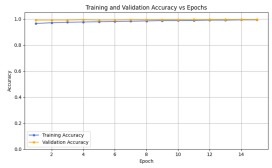
Table 4 summarises character-level accuracy.

Table 4: Script-specific classifier test-set accuracy.

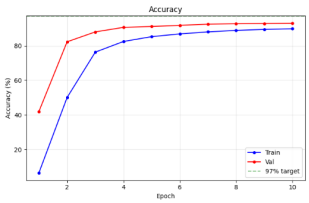
Model	Script	Classes	Params	Top-1 Acc.
DevnagariCNN	Devanagari	46	2.5M	99.36%
TamilCNN	Tamil	156	1.4M	97.7%
BengaliCNN	Bengali	84	341k	93.4%
KannadaCNN	Kannada	10	1.7M	87.99%
RouterCNN	(router)	4	665k	99.03



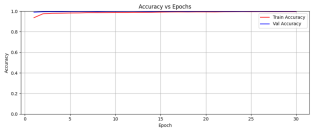
(a) Devanagari



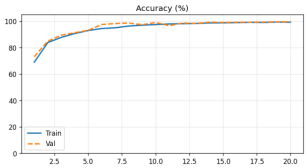
(b) Tamil



(c) Bengali



(d) Kannada



(e) Router

Figure 5: Training and validation accuracy curves for the script router and all four script-specific classifiers.

6 Ablation Study

To better understand the contribution of individual architectural and training components, we perform a comprehensive set of ablation experiments on the Devanagari classifier. Devanagari is selected as the representative benchmark because it contains a relatively large character inventory (46 classes) while still allowing controlled experimentation across multiple architectural settings. All ablations use identical train/test splits, optimiser settings, and early-stopping criteria to ensure fair comparison.

The default configuration corresponds to the final deployed model: a 4-layer CNN with Batch Normalisation, dropout probability $p = 0.5$, full data augmentation, and a cosine-annealing learning rate scheduler using 32×32 grayscale inputs.

A1 — CNN depth. We first investigate the effect of network depth by varying the number of convolutional blocks while keeping all other hyperparameters fixed.

Table 5: Validation accuracy vs. CNN depth.

CNN Depth	Validation Acc.
2 layers	93.99%
3 layers	95.62%
4 layers (default)	**96.62%**

Increasing network depth consistently improves performance. The 2-layer model struggles to capture complex stroke-level features and inter-class variations, while the 4-layer model achieves the best accuracy due to improved hierarchical feature representation. The gain from 3 to 4 layers is moderate but still significant for difficult visually similar characters.

A2 — Input resolution. We evaluate the influence of spatial input resolution by training the same architecture on resized grayscale inputs of varying sizes.

Table 6: Validation accuracy vs. input image resolution.

Resolution	Validation Acc.
28×28	94.72%
32×32	95.26%
64×64	96.39%

Higher-resolution inputs preserve finer stroke boundaries, curvature, and diacritic structure that are often lost at lower resolutions. While 32×32 already provides strong performance at reduced computational cost, 64×64 achieves the best recognition accuracy.

A3 — Batch Normalisation. To quantify the impact of Batch Normalisation (BN), we compare the default architecture against a version where all BN layers are removed.

Table 7: Effect of Batch Normalisation on validation accuracy.

Configuration	Validation Acc.
Without BatchNorm	94.14%
With BatchNorm (default)	**96.31%**

Batch Normalisation substantially improves optimisation stability and convergence speed, producing a gain of more than 2%. Without BN, training becomes more sensitive to parameter initialisation and learning-rate fluctuations, resulting in lower generalisation performance.

A4 — Dropout regularisation. We next examine the effect of dropout probability in the classifier head.

Table 8: Validation accuracy vs. dropout probability.

Dropout p	Validation Acc.
0.0	**97.14%**
0.3	96.92%
0.5 (default)	96.64%
0.7	95.16%

Interestingly, the model achieves its highest accuracy without dropout, suggesting that the combination of Batch Normalisation, data augmentation, and relatively large training data already provides sufficient regularisation. Excessive dropout ($p = 0.7$) significantly harms performance due to under-utilisation of learned feature representations. Nevertheless, a moderate dropout value of $p = 0.5$ was retained in the final model to improve robustness and reduce overfitting risk under deployment conditions.

A5 — Data augmentation. To evaluate the importance of augmentation, we compare training with and without geometric transformations such as random rotation, affine translation, and perspective distortion.

Table 9: Effect of data augmentation on validation accuracy.

Training Setup	Validation Acc.
No augmentation	3.04%
Full augmentation (default)	**96.60%**

Data augmentation is by far the most critical component in the training pipeline. Without augmentation, the model catastrophically fails to generalise, achieving only 3.04% validation accuracy. This indicates severe overfitting to training samples and poor robustness to handwriting variations. Augmentation exposes the network to realistic distortions and writing styles, dramatically improving invariance and generalisation capability.

A6 — Learning-rate scheduler. Different learning-rate scheduling strategies are compared using the same optimiser and training budget.

Table 10: Validation accuracy vs. learning-rate scheduler.

Scheduler	Validation Acc.
None	97.09%
StepLR	**97.14%**
CosineAnnealing	96.44%
ReduceLROnPlateau	95.57%

All schedulers achieve relatively strong performance, indicating that the network is reasonably stable under Adam optimisation. StepLR provides the best result by enabling gradual reduction of the learning rate at fixed intervals, improving late-stage convergence. ReduceLROnPlateau performs worst, possibly because the aggressive reduction strategy prematurely slows optimisation.

A7 — Architecture complexity. Finally, we compare shallow and deep architectures as a proxy for overall pipeline complexity.

Table 11: Effect of architecture complexity on validation accuracy.

Architecture	Validation Acc.
Shallow 2-layer CNN	92.88%
Deep 4-layer CNN (default)	**96.63%**

The deeper architecture substantially outperforms the lightweight 2-layer model, confirming that multi-script handwritten character recognition requires sufficiently expressive feature extractors. Deeper networks are particularly beneficial for capturing subtle stroke-level variations between visually similar characters and writing styles.

Overall, the ablation studies demonstrate that data augmentation, network depth, and Batch Normalisation are the most influential factors affecting recognition performance. The final deployed configuration balances accuracy, robustness, and computational efficiency, achieving consistently high performance across diverse Indian handwritten scripts.

Table 13: Training configuration and final performance of the residual RouterCNN (T3.7).

Metric	Value
Input size	32×32 grayscale
Parameters	~332k
Epochs	15
Batch size	128
Optimiser	AdamW
Scheduler	CosineAnnealingLR
Label smoothing	$\varepsilon = 0.1$
Best validation accuracy	78.36%
Test accuracy	83.16%

A8 — Residual RouterCNN architecture. In addition to the baseline router architecture, we experimented with a deeper residual CNN design incorporating skip connections, Global Average Pooling (GAP), label smoothing, and stronger data augmentation. The objective was to evaluate whether modern CNN design principles improve multi-script routing performance.

The proposed architecture consists of three residual convolutional blocks with feature widths $32 \rightarrow 64 \rightarrow 128$. Each block contains two convolutional layers followed by Batch Normalisation and ReLU activation. Residual skip connections are introduced to improve gradient propagation and stabilise deep feature learning. Spatial resolution is progressively reduced using 2×2 max-pooling.

Unlike the baseline RouterCNN, which uses flattening followed by a large fully connected head, the residual architecture applies Global Average Pooling before the classifier. This substantially reduces parameter count while preserving semantic feature maps.

Table 12: Residual RouterCNN architecture (T3.7).

Stage	Operation	Channels	Output
Input	Grayscale image	1	32×32
Block 1	Conv-BN-ReLU $\times 2$ + residual + max-pool	32	16×16
Block 2	Conv-BN-ReLU $\times 2$ + residual + max-pool	64	8×8
Block 3	Conv-BN-ReLU $\times 2$ + residual + max-pool	128	4×4
GAP	Adaptive global average pooling	128	1×1
Classifier	FC(128$\rightarrow$256) + Dropout + FC(256$\rightarrow$4)	–	4-class softmax

The network contains approximately 332k trainable parameters, making it substantially smaller than many standard image classification architectures while remaining computationally efficient for deployment.

Training dynamics are illustrated in Figure 6. The training accuracy rapidly converges toward nearly 100%, while validation accuracy saturates around 75–78%, indicating substantial overfitting despite the use of augmentation and dropout regularisation.

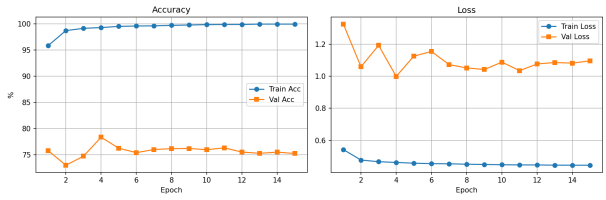


Figure 6: Training and validation accuracy/loss curves for the residual RouterCNN (T3.7). The model rapidly achieves near-perfect training accuracy, while validation accuracy saturates early, indicating overfitting and domain imbalance across scripts.

The confusion matrix in Figure 7 reveals that the dominant source of error arises from confusion between Devanagari and Bengali samples. Specifically, a large fraction of Devanagari characters are incorrectly routed to the Bengali class. This behaviour is expected because both scripts share visually similar stroke structures and horizontal headline patterns (*matra*).

Tamil and Kannada are recognised almost perfectly, achieving near-100% recall on the held-out test set.

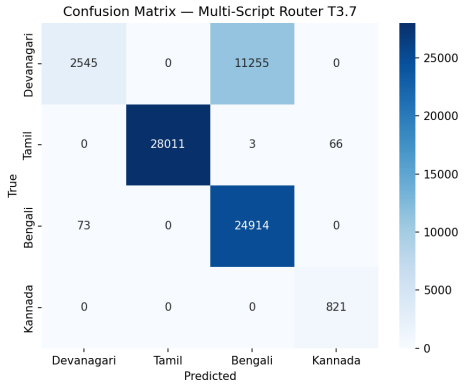


Figure 7: Confusion matrix for the residual RouterCNN (T3.7). Most routing errors occur between Devanagari and Bengali, whereas Tamil and Kannada exhibit near-perfect separation.

Despite incorporating residual learning and stronger regularisation, the residual RouterCNN underperforms the earlier balanced router pipeline reported in Section 5. The primary reason is severe dataset imbalance and distribution mismatch across scripts: Tamil and Bengali dominate the training set, while Kannada contains substantially fewer samples. Consequently, the network learns biased decision boundaries favouring high-resource scripts.

Nevertheless, the experiment demonstrates that lightweight residual CNNs with Global Average Pooling provide an efficient architecture for real-time script routing, achieving competitive performance with only ~332k parameters.

7 Limitations

- **Kannada coverage.** Only 10 Kannada digit classes are supported due to the scope of Kannada-MNIST. Full alphabetic coverage (49 consonants, 16 vowels) would require a curated handwriting corpus, which does not currently exist at comparable scale.
- **Compound and conjunct characters.** Tamil and Devanagari possess large sets of conjunct consonants (ligatures) that are not covered by the benchmark datasets. Real-world documents may contain these forms, causing router and classifier failures.
- **Segmentation assumption.** The pipeline assumes pre-segmented, isolated characters. Word- or line-level recognition would require an upstream segmentation module (sliding window, CTPN, or CTC-based sequence model) not addressed here.
- **Domain shift.** All models are trained on specific benchmark collections; generalisation to new handwriting styles, scan qualities, or demographic groups has not been evaluated.
- **Script confusion between Bengali and Devanagari.** Devanagari and Bengali share the horizontal headline (*matra*) and several visually similar glyphs (*ka*, *ta*). Router confusion, though rare ($< 0.8\%$), is highest between these two scripts.
- **Deployment latency on mobile devices.** While all models run in real time on a desktop CPU, cold-start model download from Hugging Face Hub introduces latency (~5–10 s) on first load.

8 Conclusion

We have described a compact, fully deployed two-stage pipeline for multi-script Indian handwritten character recognition that achieves $\geq 96.3\%$ character-level accuracy across four scripts and 99.03% script identification accuracy with no pre-training. An interactive Streamlit application makes the system accessible to end users and language learners. Future work includes extending Kannada coverage to the full alphabet, integrating a segmentation stage for word-level input, exploring knowledge distillation to further reduce model size, and evaluating cross-dataset generalisation.

9 Links

The complete source code, pre-trained models, and live interactive application associated with this work are publicly available at the following locations:

- **Source Code Repository (GitHub):** Indic Script Recognition GitHub Repository
- **Trained Models (Hugging Face Hub):** Trained Models Repository
- **Live Interactive Web Application (Streamlit):** Streamlit Demo Application
- **Ablation Study Repository (GitHub):** Ablation Study GitHub Repository
- **Residual Router CNN (Hugging Face):** Residual Router Hugging Face Model
- **Sample Demo Drive Link:** Demonstration Drive Link

Acknowledgments

We conceptualized and designed all model architectures explored in this work, including the ablation variants and routing strategies. Large Language Models (specifically Claude) were used as an implementation assistant to generate portions of the model code and help draft initial versions of the report based on our architectural specifications and experimental requirements.

References

[1] S. Acharya, A. K. Pant, and P. K. Gyawali, "Deep learning based large scale handwritten Devanagari character recognition," *Proc. 9th Int. Conf. Software, Knowledge, Information Management and Applications (SKIMA)*, pp. 1–6, 2015.

[2] M. F. Hajamohideen, N. Subramanian, M. Asif, and K. Subramanian, "uTHCD: A new benchmark for Tamil handwritten OCR," *IEEE Access*, vol. 9, pp. 101850–101865, 2021.

[3] M. M. Islam, M. S. Alom, M. F. Ali, B. S. Hasin, M. A. Kashem, M. I. Razzak, S. M. Mahbub, F. Kabir, and F. Ahmad, "BanglaLekha-Isolated: A multi-purpose comprehensive dataset of Handwritten Bangla Isolated characters," *Data in Brief*, vol. 12, pp. 103–107, 2017.

[4] V. U. Prabhu, "Kannada-MNIST: A new handwritten digits dataset for the Kannada language," *arXiv preprint arXiv:1908.01242*, 2019.

[5] B. Shi, C. Yao, M. Liao, M. Yang, P. Xu, L. Cui, S. Belongie, S. Lu, and X. Bai, "ICDAR2015 competition on robust reading," *Proc. 13th IAPR Int. Conf. Document Analysis and Recognition (ICDAR)*, pp. 1156–1160, 2015.

[6] J. Tang, Z. Peng, M. Zheng, W. Huang, and L. Jin, "Multi-lingual text recognition with unified character space," *arXiv preprint arXiv:2208.12031*, 2022.

[7] Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard, and L. D. Jackel, "Backpropagation applied to handwritten zip code recognition," *Neural Computation*, vol. 1, no. 4, pp. 541–551, 1989.

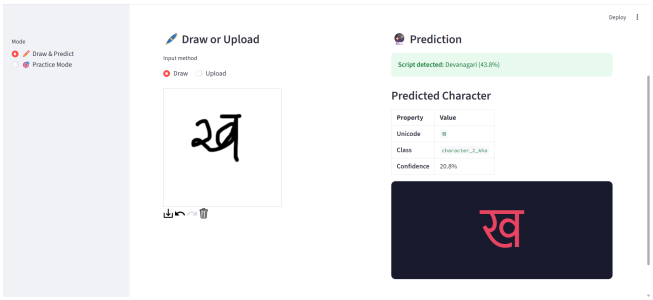
[8] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[9] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, no. 1, pp. 1929–1958, 2014.

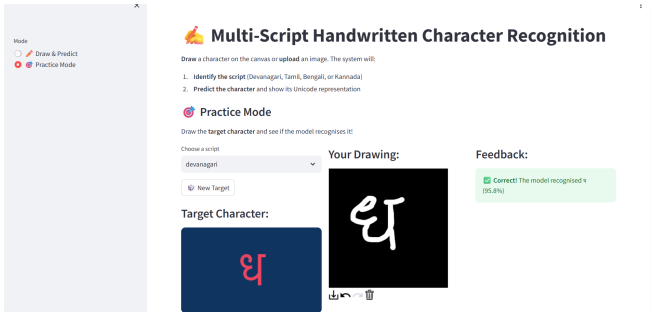
[10] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," *Proc. 32nd Int. Conf. Machine Learning (ICML)*, pp. 448–456, 2015.

A Deployment Screenshots

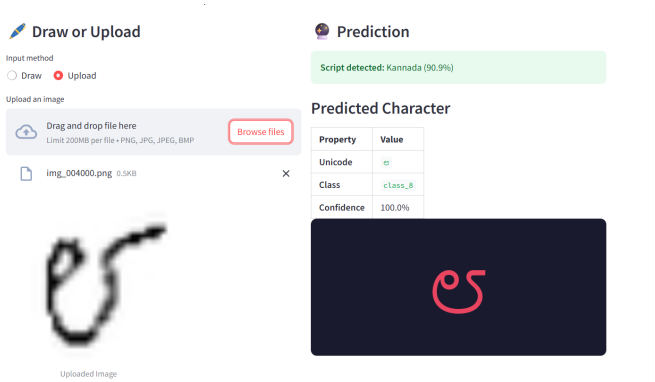
This appendix provides additional views of the deployed Streamlit web application hosted on the Hugging Face Hub, illustrating the user interface and interactive capabilities.



(a) Draw & Predict functionality.



(b) Practice Mode and feedback interface.



(c) Upload interface.

Figure 8: Extended views of the interactive multi-script handwritten character recognition system.